

❗ Ez a kvíz újra lett értékelve, a pontszáma nem változott.

Minta Beugró

Határidő Nincs megadva határidő

Pont 12

Kérdések 12

Időkorlát Nincs

Instrukciók

Minta beugró a géptermi zárthelyi előtti beugróhoz.

A beugróban a mintához hasonló felépítésű kérdések lesznek. Érinthetnek a félév folyamán vett bármely elméleti vagy gyakorlati témát, olyat is, amit a minta nem tartalmaz.

A tényleges beugró 20 kérdést fog tartalmazni. A megírására 30 perc fog rendelkezésre állni. A sikerességéhez a kérdések 60%-ra (azaz 12 kérdésre) kell helyesen válaszolni.

Próbálkozások naplója

	Próbálkozás	Idő	Eredmény	Újraértékel
LEGUTOLSÓ	1. próbálkozás	7 perc	8 az összesen elérhető 12 pontból	8 az összesen elérhető 12 pontból

Beadva ekkor: máj 17, 01:33

1. kérdés

0 / 1 pont

Az alábbi kifejezés esetén milyen programozási tételt alkalmazunk?

$$r := \bigotimes_{\substack{e \geq 2 \\ e \in I \\ \text{even}(e)}} f(e)$$

legadott válasz

☒ Ez egy hibás jelölés

helyes válasz

☐ Feltételes összegzés az első 2-nél nagyobb elemig

☐ Feltételes maximum kiválasztás, tudva, hogy minden elemre igaz, hogy $\text{even}(e)$

☐ Lineáris keresés két feltétellel

2. kérdés

1 / 1 pont

Milyen SOLID elvet sértünk akkor, amikor olyan osztályokat készítünk, amik a saját hatáskörüket túllépik, és több működést foglalnak magukba, mint amennyit kellene.

Helyes!

- ☒ Single Responsibility Principle
- ☐ Liskov Substitution Principle
- ☐ Interface Segregation Principle
- ☐ Dependency Inversion Principle

3. kérdés

1 / 1 pont

Az alábbiak közül melyik módon NEM lehet függőség befecskendezést elérni?

- ☐ Osztálysablon alkalmazása
- ☐ Objektum befecskendezés
- ☐ Származtatás
- ☒ Láthatóság korlátozása

Helyes!

4. kérdés

Eredeti eredmény: 1 / 1 pont Újraértékelt eredmény: 1 / 1 pont**! Ez a kérdés újra lett értékelve.**

Milyen problémát old meg a Visitor (Látogató) tervezési minta?

legadott válasz

Lehetővé teszi bejárható adattípusok elemeinek foreach jellegű bejárását, "meglátogatva" őket, ezzel feleslegessé teszi a hagyományos for ciklus használatát.

helyes válasz

Lehetővé teszi, hogy a látogató objektum "meglátogasson" egy másik objektumot, ezzel információt szerezve a látogatott objektum konkrét osztályáról, amelyre azért lehet szüksége, mert a látogató viselkedése a konkrét osztálytól függ, azonban csak az őosztály/interfész referenciához fér hozzá.



Belső viselkedést maszkol, ezáltal az objektum "látogatói" csak egy maszkot látnak, a belső működést elfedi.



Két inkompatibilis interfész összekapcsolását oldja meg, az egyik interfész "látogatóként" funkcionál, a másik interfész pedig "vendégül látja", a vendégeskedés keretében limitált ideig hozzáférést biztosít egyes metódusaihoz.

5. kérdés**0 / 1 pont**

Mi a különbség egy UML osztálydiagramon az absztrakt osztály és egy interfész között?



Az interfész az osztály része, nem egy külön típus, míg az absztrakt osztály önálló típus.



Nincs különbség, a kettő lényegében megegyezik, csak nem minden OOP nyelv támogatja ezeket, így van lehetőség specifikus jelölésre.

Ielyes válasz



Az interfész neve fölé oda van írva az <<interface>> jelző, míg az absztrakt osztályt a dőlt név jelöli.

Iegadott válasz

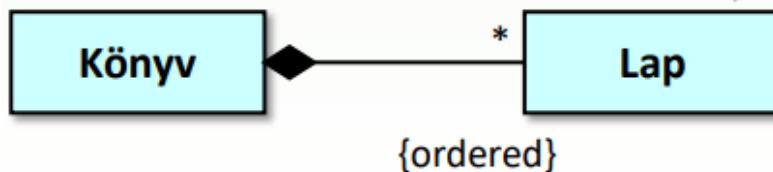


Az interfésznek nincsen metódus törzse, se adattagjai.

6. kérdés

0 / 1 pont

Mi igaz a Könyv és Lap osztályokra, ha az alábbi kapcsolat van közöttük?



Iegadott válasz



Egy Könyv akármennyi lapot tartalmazhat, hiszen a csillag akár a 0-t is megengedi.

Ielyes válasz



Nem létezhet olyan Könyv, amihez nem tartozik legalább 1 Lap.



Egy Lap akármennyi Könyvnek része lehet.



A Könyv és a Lap közötti kapcsolat egy aggregáció.

7. kérdés**0 / 1 pont**

Adott egy Robot osztályunk a következő konstruktorral. Szeretnénk egy másik konstruktort is megadni, ami egy másik Robotot ad. Hogyan adjuk ezt meg?

Robot(int s) { sorozatszam = s; }

Ielyes válasz

☐ Robot(Robot r) : this(r.sorozatszam + 1) { }

☐ Robot(Robot r) : Robot(r.sorozatszam + 1) { }

Iegadott válasz

☒ Robot(Robot r) { Robot(r.sorozatszam + 1); }

☐ Ez nem lehetséges C#-ban.

8. kérdés**1 / 1 pont**

Melyik állítás igaz?

Helyes!

☒ Lehet olyan osztályt használni, melyet nem lehet példányosítani.

☐ Leszármazás esetén nem szükséges a virtuális függvényeket megjelölni, mivel alapértelmezetten a OOP nyelvek minden függvényt virtuálisnak tekintenek.

☐ Egy kapcsolat esetén annak minden érintett osztálya tartalmazni fog egy adattagot a másik osztályokról.

☐ Sose veszünk fel protected adattagot.

9. kérdés**1 / 1 pont**

A tervben egy metódus két értéket ad vissza, hogyan lehet ezt megvalósítani?

Helyes!

- ☐ Referenciaként átadott paraméterekkel
- ☒ A többi eset mind lehetséges
- ☐ Két out paraméter segítségével
- ☐ tuple-t visszaadva

10. kérdés**1 / 1 pont**

A következő osztálydefiníciók alapján, melyik program sorok lesznek helyesek?

```
class Rovar { }
```

```
class Bogár : Rovar { }
```

```
class Futrinka : Bogár { }
```

Helyes!

- ☐ Rovar r = new Bogár();
- ☐ Bogár b = (Futrinka) r;
- ☐ Rovar r = new Bogár();
- ☐ Futrinka f = (Futrinka) r;
- ☐ Rovar r = new Futrinka();
- ☒ Bogár b = (Futrinka)r;
- ☐ Rovar r = new Rovar();
- ☐ Futrinka f = (Futrinka) r;

11. kérdés**1 / 1 pont**

Mi történik, ha le hagyjuk az override kulcsszót egy virtuális metódus felülírásakor?

☐

Így átfolyhat más megegyező paraméter listájú metódusokra, azokat is felülírva.

☐

Semmi, ezt kitenni opcionális.

☒

Bizonyos esetekben az őszosztály (felülírni szánt) metódusa futhat le.

☐

Fordítási hibát kapunk.

Helyes!**12. kérdés****1 / 1 pont**

MSTest környezetben mire használjuk a következő programsort?
Assert.AreEqual(42, life.Meaning);

☒

Megvizsgáljuk, hogy a life.Meaning egyenlő-e 42-vel.

☐

Meghívjuk a life.Meaning-et 42-ször és megvizsgáljuk, hogy minden híváskor ugyanaz-e az értéke.

☐

Addig felfüggesztjük a tesztet, amíg a life.Meaning értéke 42 nem lesz.

☐

Felülírjuk a life.Meaning viselkedését, hogy a teszt lefutása alatt végig 42 legyen.

Helyes!

